

Internship Application Copilot

SUMMARY

Automatically understand and fill internship/job application forms using a user's single reusable professional profile.

DOCUMENT METADATA

Document Version: 1.0 Status: Development Baseline Target: Web Application + Chrome Extension + AI Backend

PRODUCT VISION

Internship Application Copilot is a browser-integrated application that allows a user to create their professional profile once and reuse it across internship and job applications.

CORE PRODUCT PRINCIPLE

The application must NOT be built around individual websites. Instead, the system must understand semantic meaning and map fields to canonical profile attributes.

RECOMMENDED ARCHITECTURE

Web Dashboard ↔ Backend API ↔ PostgreSQL + AI Provider, with Chrome Extension interacting with internship websites for DOM scanning, field detection, and autofill.

TECHNOLOGY STACK

Frontend: Next.js, TypeScript, App Router, Tailwind CSS, shadcn/ui, React Hook Form, Zod. Extension: Chrome Extension (Manifest V3), TypeScript, React, Vite. Backend: Next.js server, route handlers/server actions, PostgreSQL, Prisma ORM. Storage: AWS S3 or S3-compatible object storage. Authentication: Auth.js / OAuth (Google, Email).

PROJECT STRUCTURE

Monorepo with apps/web, apps/extension, shared packages (database, types, validation, ai, config), tests, docs, and workspace tooling.

PROFILE SYSTEM

Profile sections include Personal, Education, Professional, Links, Skills, Projects, Experience, Achievements, and Certifications with normalized internal schema paths.

FORM DETECTION & FIELD MAPPING

Inspect input/textarea/select/button/label/fieldset/radio/checkbox/file/contenteditable, extract metadata, run deterministic matching first, then AI fallback with confidence scoring.

AI STRATEGY

Use specialized operations: Field Classifier, Job Description Analyzer, Resume Matcher, Answer Generator, Profile Normalizer. Use tiered models with rule engine for known deterministic fields.

PRIVACY & SECURITY

Data minimization, no storage of passwords/OTP/bank/credit data, validate AI outputs through JSON + Zod + business rules, least-privilege permissions, and secure backend practices.

DEVELOPMENT PHASES

Phase 0 to Phase 8 covering architecture, profile system, extension basics, deterministic autofill, AI mapping, dynamic applications, resume intelligence, answer assistant, and application tracking.

MVP SCOPE

Include: account creation, profile creation, resume upload, Chrome extension, form detection, basic autofill, AI field mapping, confidence scoring, review screen, application history. Exclude automatic submission and advanced automation.

ARCHITECTURAL RULEBOOK

Key rules include: extension owns browser interaction, backend owns business logic, deterministic rules before AI, AI never directly controls browser, low-confidence never silently filled, and no automatic submission in MVP.

DEFINITION OF DONE

Complete when profile, resumes, extension auth, form detection/normalization, deterministic + AI mapping, confidence handling, dynamic/multi-step support, review flow, validation, testing, and deployability are all in place.